

Power BI DAX and Dynamic Report



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://about.softuni.bg/>

1. Report Design Strategy (Audience & Story)
2. Navigation & Layout
3. Introduction to DAX (The Engine)
4. Time Intelligence (YoY Growth)
5. Advanced Analytics (Forecast, Custom Visuals)



The "Report Factory" Process

Don't just start clicking visuals! Follow the process:

1. Requirement Gathering: What problem are we solving?
2. Data Prep: SQL / Power Query.
3. Modeling: Star Schema.
4. DAX: Business Logic & KPIs.
5. Visuals: The Final Output.



Planning for a Report

Good planning makes reports readable, relatable, and reliable.

Know Your Audience

Executives

Wants KPIs, Trends, High-level summary, No tables, Clean visuals

Analysts/Manager

Detailed tables, Wants Slicers, Filtering, Ability to drill down

Operations

Real-time data, Alerts, Detailed lists

Define Objectives

What questions should the report answer?

What actions should it drive?

Match chart types to data and goals.

Plan Navigation

Interactivity

bookmarks & buttons

page layouts

Consider mobile vs. desktop experience.



A good story in a Report answers:

- What's happening?
- Why is it happening?
- What should we do about it?

Visual hierarchy

Consistency

Whitespace

Color Usage

Report Sections

Key Design
Principles

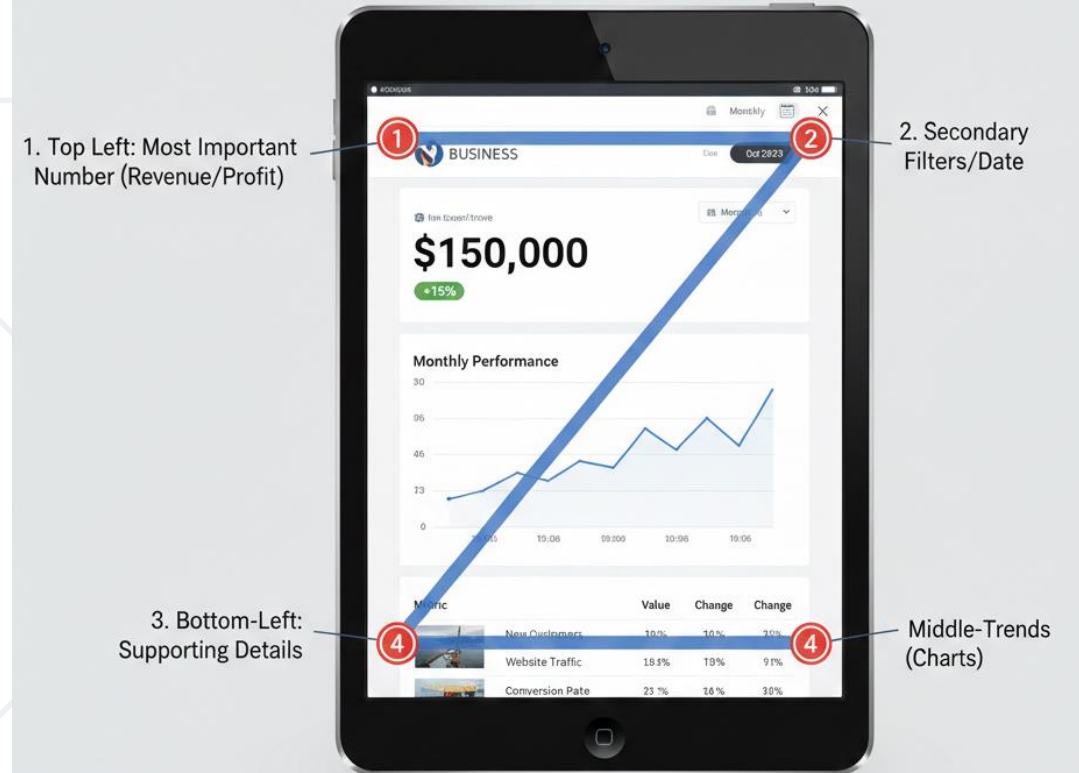


Layout Principles (The Z-Pattern)

Diagram showing eye movement 'Z' across screen

Rule:

1. Top Left: Most important number (Revenue/Profit).
2. Top Right: Secondary Filters/Date.
3. Middle: Trends (Charts).
4. Bottom: Details (Tables).



Visual hierarchy

Key Design Principles

Consistency

Whitespace

Color Usage

Report Sections

1. Why Visual Hierarchy is important?

- Focus on key insights quickly
- Improves data storytelling and decision-making
- Helps on user to prioritizing information

2. How to use Visual Hierarchy effectively?

- Bigger elements attracts the attention first
- Size is used for underlining important data
- Put key elements at the top based on the natural reading flow
- Use bigger font size for key values and section titles
- Combine the use of position, color and size for maximum effect
- Remember that less is more

Consistency

Visual hierarchy

Whitespace

Color Usage

Report Sections

1. Why Consistency is important?

- Build trust and feeling of being familiar with the users
- Easier navigation and understanding
- Optimized cognitive load by following patterns

2. How to maintain Consistency

- The same alignment and grid across pages
- Navigation in the same position across multi-page reports
- Same or similar types of charts for similar data
- Harmonious color pallet across visuals
- Uniform color usage meaning
- Reuse of design templates or style guide
- Ensure good UX across different types of devices

Whitespace

Visual hierarchy

Consistency

Color Usage

Report Sections

1. Why Consistency is important?

- Clarity and readability
- Allows to create focus points/areas
- Reduce clutter overload

2. How to use Whitespace effectively

- Use free space between visuals
- Optimal grouping
- Equal paddings and margins
- Whitespace allows users to natural flow of reading/perceiving
- Create a balance – do not overlap
- Use between titles, sub-titles and values
- Grid option could be helpful
- Use containers to maintain space

Color Usage

Visual hierarchy

Consistency

Whitespace

Report Sections

1. Why Color Usage is important?

- Attracts attention to key data
- Strengthens the understanding and interpretation
- Creates context

2. How to use Colors effectively

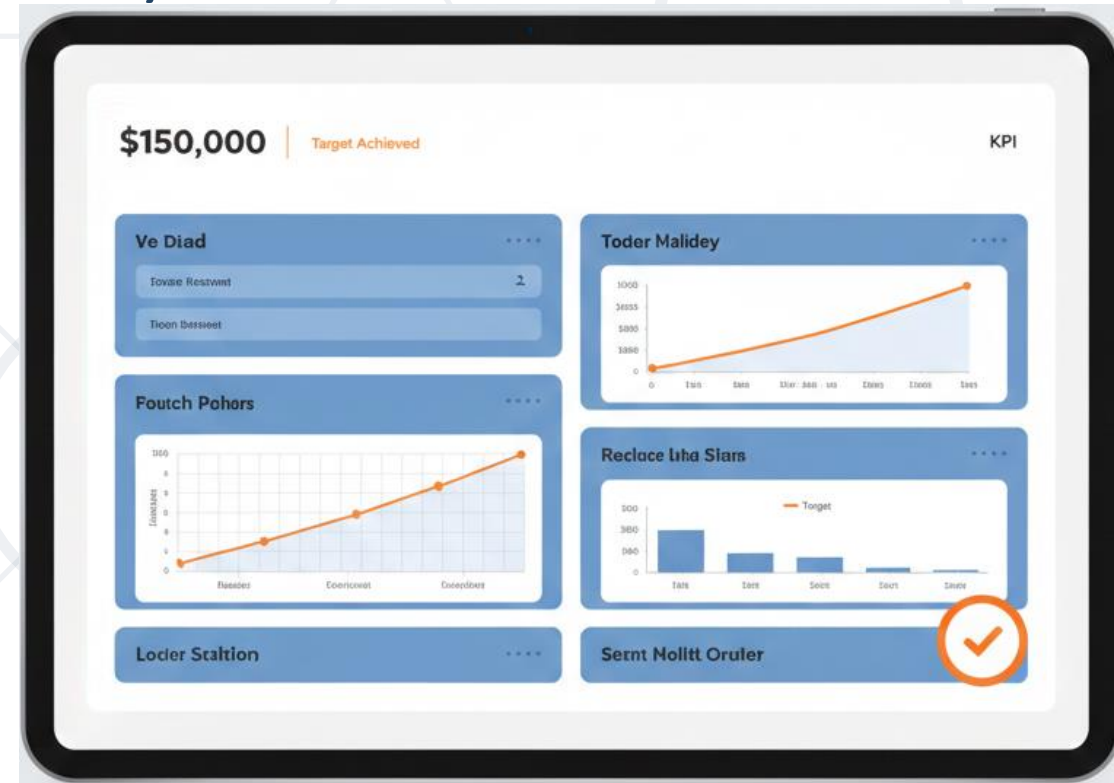
- Use bright or contrasting colors for underlining or stressing on important data
- Do not overuse bright colors – use only for accents
- Use standard color pallet – based on a project/company
- One and the same colors for the same categories
- Low contrasting text and visual is hard to read (white and light grey)
- Use icons or text as enhancement
- Use legends for color meaning when appropriate
- COLOR should be used intentionally not for esthetics

60-30-10 Rule:

- 60% Neutral (Background, White space).
- 30% Secondary (Grey, Soft Blue).
- 10% Accent (Bright Color for Data).

Warning: Don't use Red/Green unless it's Good/Bad.

Color blind users can't see the difference.



Report Sections

Key Design Principles

Visual hierarchy

Consistency

Whitespace

Color Usage

1. Why Report Section is important?

- Improves navigation and orientation of end user
- Helps user finds information more efficiently
- Supports the logical flow of the report/analysis

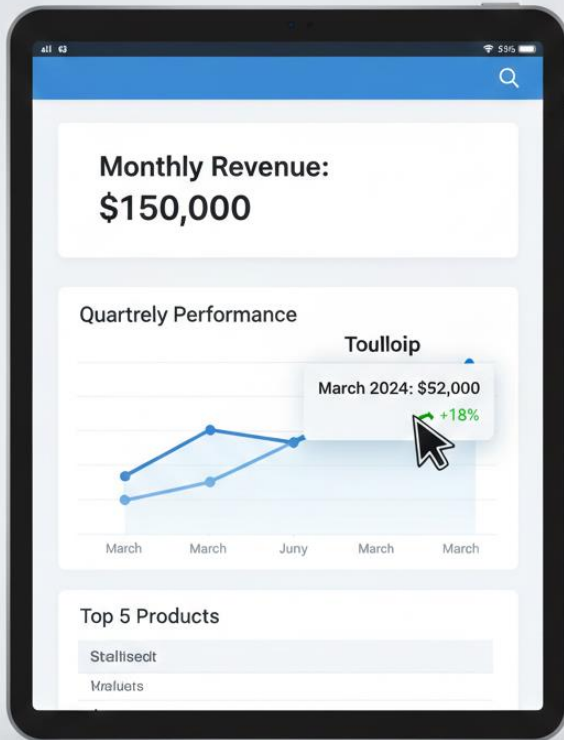
2. How to use Report Sections effectively

- Use separate pages for the different needs/stakeholder groups
- Use clear titles
- Group visualizations addressing the same question in one section
- Use clear navigation for an optimal usage of report
- The same formatting of every page in a multi-page reports/dashboards
- Add instructions or explanations for sections if needed
- Build a story from the global theme to the detailed one
- Add filters and segments for personalized analysis

Dashboard vs Report:

- Dashboard: One page summary.
- Report: Detailed multi-page document.

Navigation: Use Buttons (Page Navigator) to create an "App-like" experience.



- **Definition:** A small report that pops up when you hover over a chart.
- **Why use it:** Keeps the main page clean but provides detail on demand.



Good vs. Bad

Examples

Good example - Clean, focused, and actionable

Dealer dashboard

All

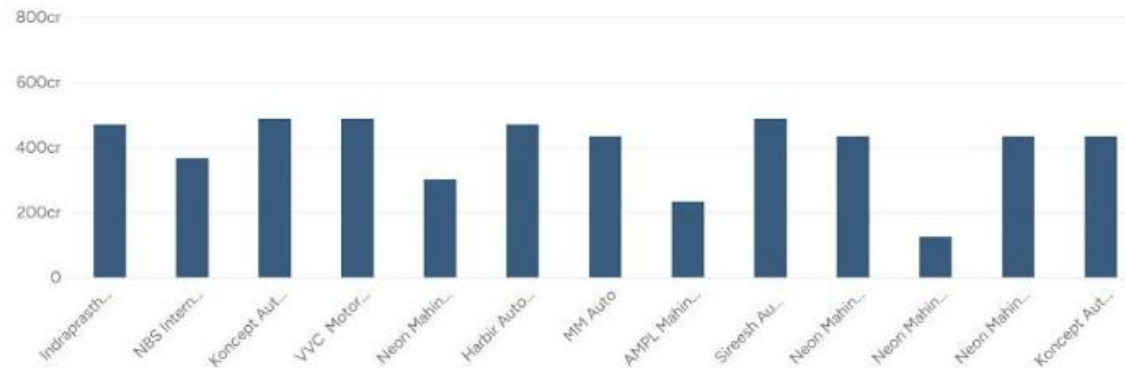
Commercial vehicles

Passenger vehicles

More

This year

Top dealers

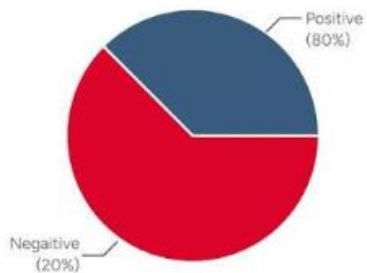


Top products



Overall customer sentiment

customer experiences and feedback sentiment



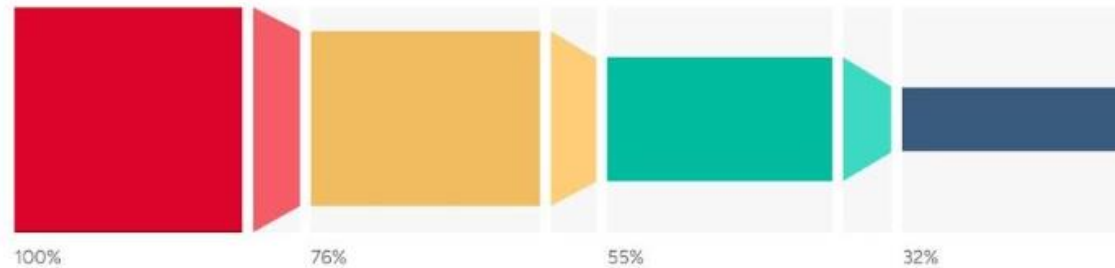
Sales funnel

Total leads
25K

Test drives
21.3K

Bookings
12.9K

Sales
5.1K



Good example - Clean, focused, and actionable



Financial Performance Dashboard

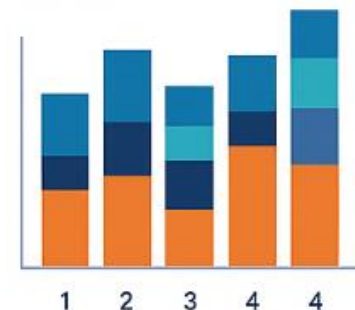
Revenue **\$250,000**

Net Profit **18.5%**

Budget vs Actual 



Expense Analytics Dashboard



Category	\$40,000
Category	\$30,000
\$4,000	\$40,000
\$25,000	\$36,000
\$30,000	\$32,000



Cash Flow Monitoring



Inflow
\$85,000

Outflow
\$75,000

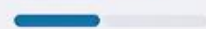


Investment Portfolio Tracker



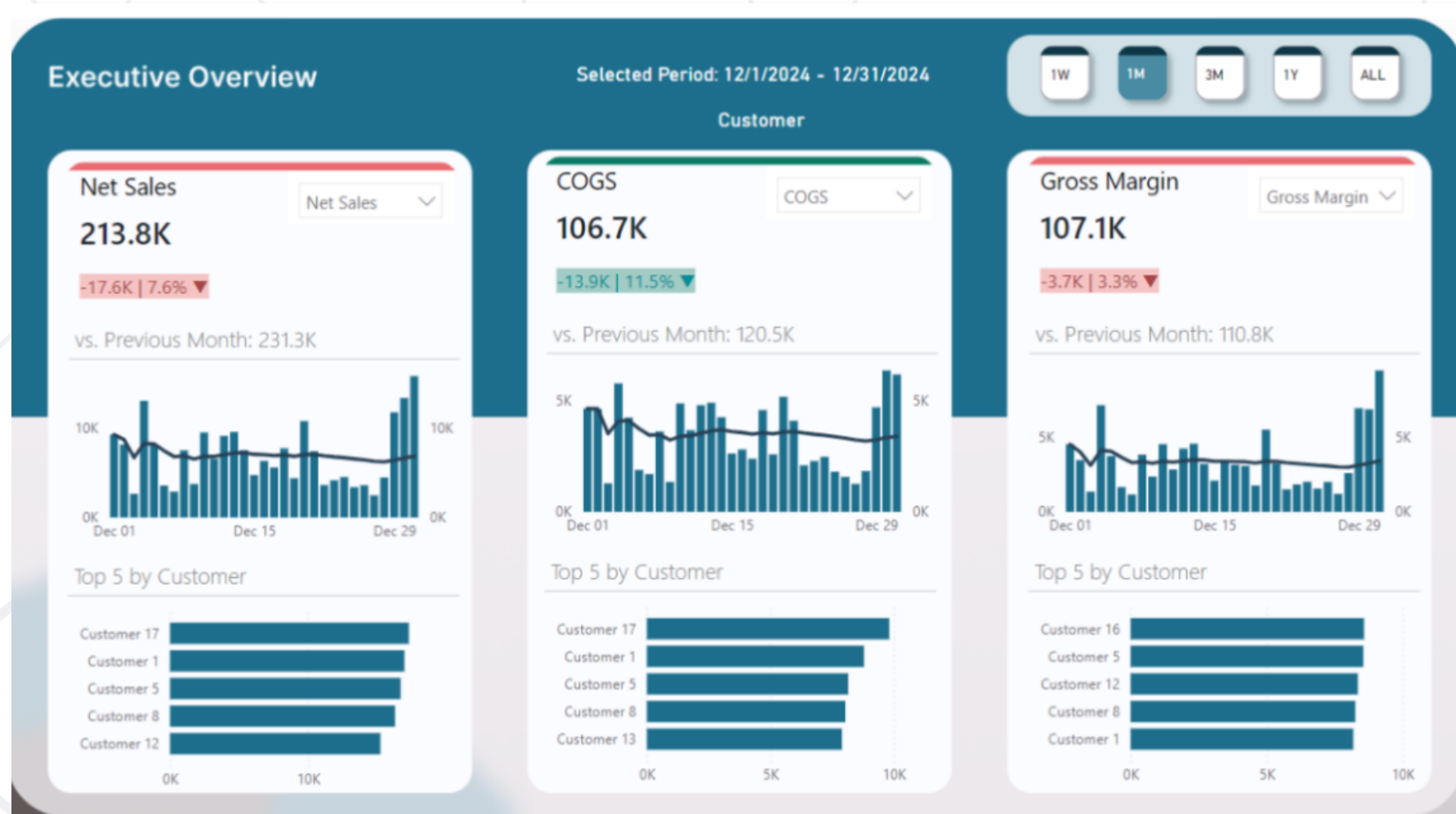
ROI **10,2%**

Risk Profile 

Asset Allocation 




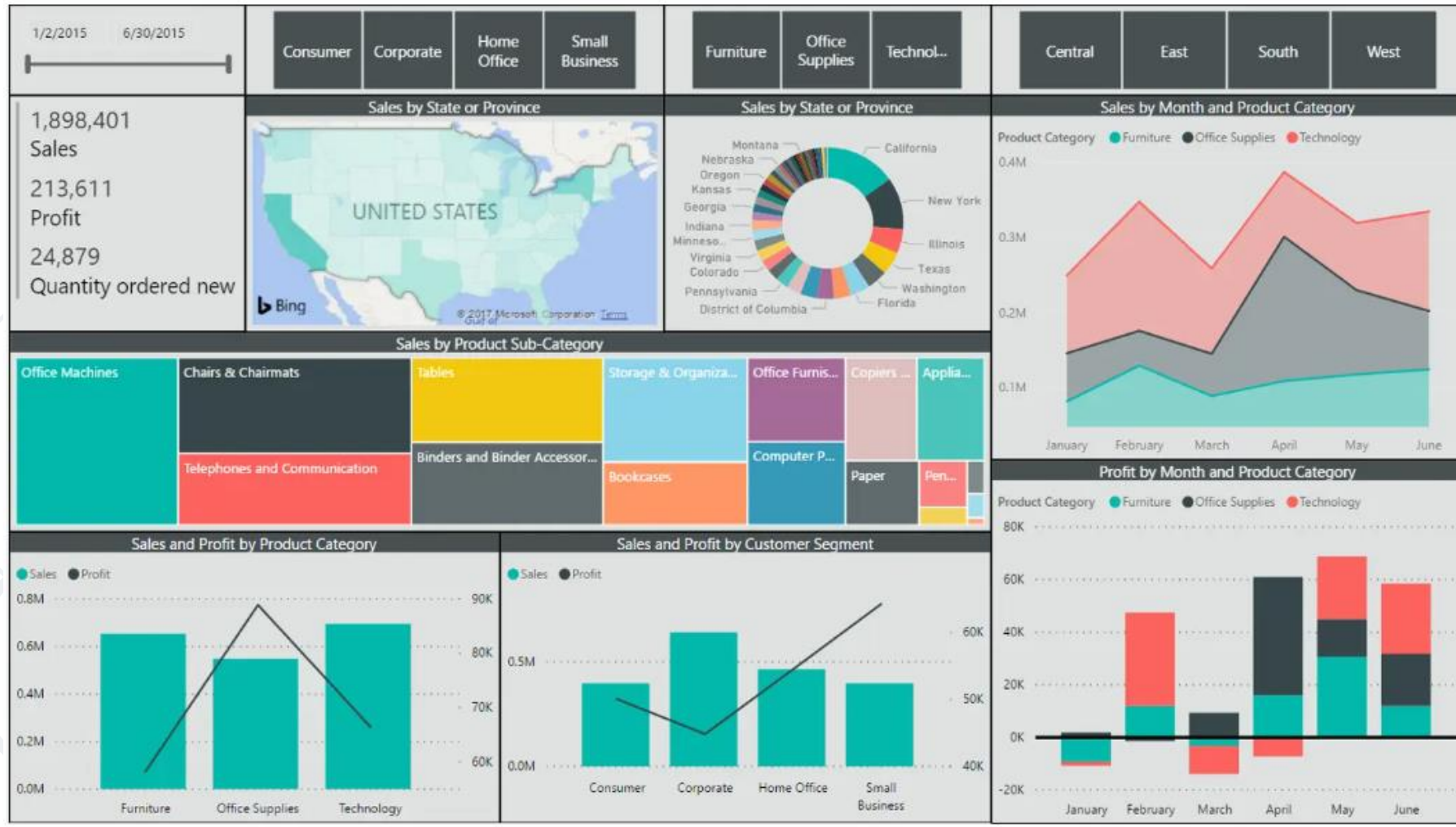
Good example - Clean, focused, and actionable



Bad example - Cluttered, confusing, and overwhelming



Bad example - Cluttered, confusing, and overwhelming



Highlights



- **Clear Visual Hierarchy:** KPIs are placed at the top, followed by trends, details, and actions.
- **Consistent Colors:** Each section uses a distinct but harmonious color.
- **Whitespace:** Adequate spacing between sections improves readability.
- **Structured Flow:** Follows a logical narrative: Overview → Trends → Details → Actions.

- **No Visual Hierarchy:** All visuals are the same size and scattered randomly.
- **Inconsistent Colors:** Distracting and uncoordinated color usage.
- **Cluttered Layout:** Too many visuals crammed into one page with no breathing room.
- **Lack of Structure:** No clear flow or storytelling—just data dumped on a canvas.



Live Demo



Data Analysis eXpressions

It looks like Excel formulas,



but it thinks differently.

Where do we use it?

- Power BI
- Excel Power Pivot
- SQL Server Analysis Services (SSAS)

Key Concept:

DAX helps you create new information from existing data.

Excel vs. DAX

Excel:

Cell-based.

`A1 + B1.`

You can change individual cells.

DAX:

Column/Table-based.

`Sales[Amount] - Sales[.]`

You apply logic to the whole dataset at once.



Columns vs. Measures

A Column is hard-coded into the table.

A Measure is a formula that waits for you to use it.

Beginners use Columns for everything. Pros use Measures for 99% of math.

Types of Calculations		
	Calculated Column  (Static)	Measure  (Dynamic)
Logic:	Calculated Row-by-Row	Calculated on Demand (Aggregation)
Storage:	Saved in RAM (Increases file size)	Uses CPU (No file size increase)
Recalculation (Update):	Only when data refreshes	Instantly when a user filters/clicks
Best For:	Slicers, Axes, Categories	Numerical Values, KPIs, Ratios
Example:	Full Name = [First Name] & " " & [Last Name]	Total Revenue = SUM(Sales[Amount])

Implicit Measures:

- When you drag a column (e.g., Amount) to a visual.
- Power BI adds the Sum (Σ) automatically.
- Problem: You cannot reuse this logic in other formulas.

Explicit Measures:

- You write the code: `Total Sales = SUM(Sales[Amount])`.
- Benefit: You can nest this inside other formulas (e.g., % of Total).
- Benefit: You can name it clearly.

Rule: Always write Explicit Measures for your core KPIs.

- **Measures:** Always refer to them as [Measure Name]. (No table name needed).
- **Columns:** Always refer to them by 'Table Name'[Column Name].

Example: Profit = [Total Sales] - SUM('Sales'[Total Cost])

Basic Aggregations

The syntax is simple: `FUNCTION( Table[Column] )`.

SUM: Adds up all numbers in a column.

`Total Revenue = SUM(Sales[Amount])`

AVERAGE: Calculates the arithmetic mean.

`Avg Price = AVERAGE(Sales[unit_price])`

MIN / MAX: Finds the smallest or largest value.

`Highest Sale = MAX(Sales[order_date])`

The "Counting" Problem

COUNTROWS: Counts how many lines are in the table (Transactions).

Total Orders = `DISTINCTCOUNT('vw_fct_sales'[order_id])`

DISTINCTCOUNT: Counts unique values only (Removes duplicates).

Unique Clients =

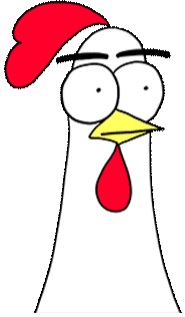
`DISTINCTCOUNT('vw_dim_customers'[customer_key])`

Scenario: Ivan bought 5 times.

`COUNT` = 5 (Wrong for client count)

`DISTINCTCOUNT` = 1 (Correct)

The "Safe Division"



#DIV/0!

The Excel Problem: In Excel, $5 / 0 = \#DIV/0!$ Error.

The DAX Solution: The **DIVIDE** function.

Syntax: **DIVIDE(Numerator, Denominator, [Alternate Result])**

Example: **Margin % = DIVIDE([Profit], [Revenue], 0)**

Adding Logic (IF)

Branching your calculation

Used to classify data dynamically.

Syntax: `IF( LogicalTest, ResultIfTrue, ResultIfFalse )`

Examples:

`Sales Status = IF( [Total Sales] > 1000, "High Performer", "Standard" )`

`Bonus Status = IF( [Total Sales] > 50000, "Bonus", "No Bonus" )`

What: Functions that loop through rows

SUMX - Sum with custom calculation per row

```
Total Revenue = SUMX(  
    Sales,  
    Sales[Quantity] * Sales[Price]  
)
```

AVERAGEX - Average with custom calc

```
Avg Margin = AVERAGEX(  
    Sales,  
    DIVIDE(Sales[Profit], Sales[Revenue])  
)
```

- When to use:
- When you need row-by-row calculation
- When simple SUM/AVG isn't enough
- Performance: Slower than simple SUM Use only when necessary!

RELATED - Navigating Relationships

What: Access columns from related tables

Use Case: Calculated Column in Fact table needs Dimension data

Example:

Sales table needs Product Category (from Product table)

Product Category (Column) =
`RELATED(Products[Category])`

How it works:

Sales → (via ProductID) → Products → Get Category

A number without a unit is meaningless

Creating the formula is only Step 1.

Step 2: Always set the format in the **Measure Tools** tab immediately.

Currency: \$ / € / BGN

Percentage: % (set decimals to 1 or 2)

Thousands Separator: (,) makes 1000000 readable as 1,000,000.

How Power BI decides which number to show

DAX formulas are dynamic. The result changes based on **Context**.

Row Context: "I am looking at the current row right now." (Used in Calculated Columns).

Filter Context: "I am looking at the data allowed by the current Slicers and Visuals." (Used in Measures).

Two Types:

1. Row Context (Calculated Columns)

"I'm looking at THIS row"

Full Name = [First Name] & " " & [Last Name]

Row 1: "Ivan" & " " & "Ivanov" → "Ivan Ivanov"

Row 2: "Maria" & " " & "Petrova" → "Maria Petrova"

2. Filter Context (Measures)

"I'm looking at FILTERED data"

Total Sales = SUM(Sales[Amount])

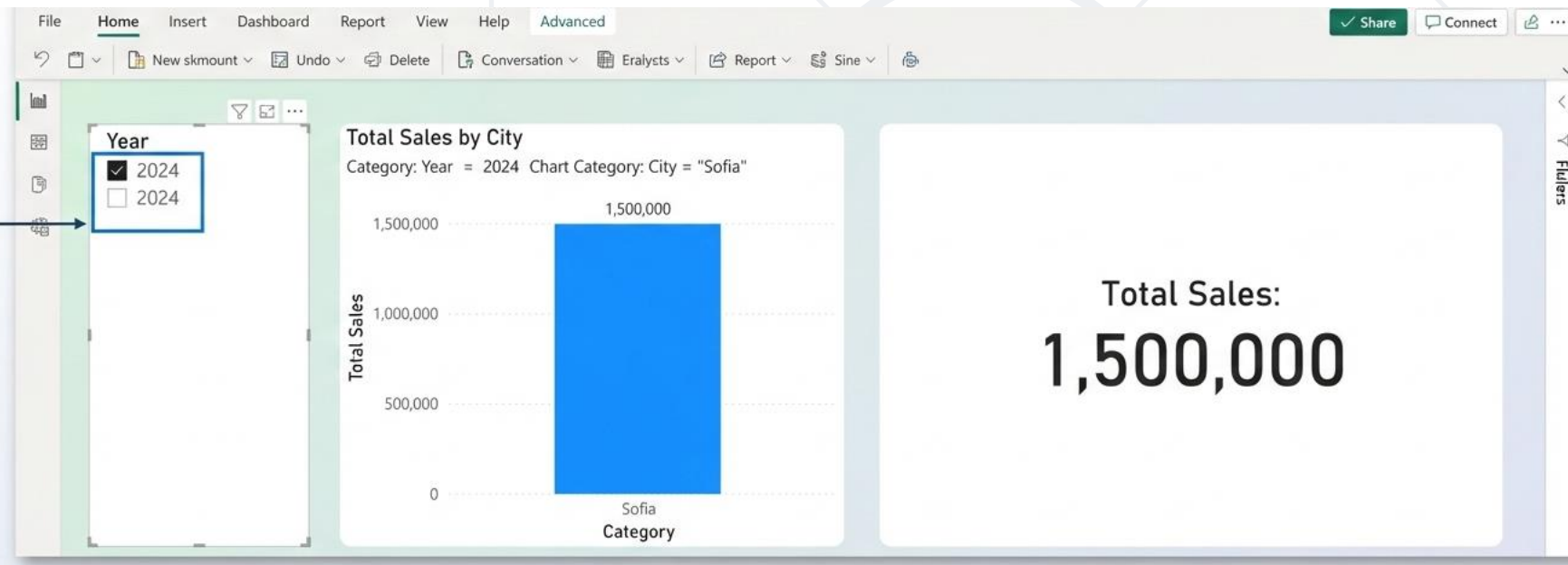
No filters: ALL sales summed

Slicer "2024": Only 2024 sales summed

Chart "Sofia": Only Sofia sales summed

Measures are SMART - they adapt to filters! Columns are DUMB - they calculate once!

Why the numbers change



The Calculation: The Measure only sums up rows where

Year=2024 AND City=Plovdiv.

It ignores everything else.

Scenario:

- Measure:

Total Sales =
`SUM(Sales[Amount])`

- Slicer:

- Year = 2024

- Chart Category:
City = "Sofia"

Live Demo



Sometimes we want to ignore the Slicer (e.g., compare 2024 vs 2023).

To do this, we need a special function that can **manipulate Context**.

If you learn only one function, learn this one.

- **Definition:** Changes the context of a calculation. The only function that can modify the Filter Context.
- **Logic:** "Calculate this number, BUT pretend the filters are different."

CALCULATE(

<Expression> , -- What to calculate (e.g., Total Sales)

<Filter 1>, -- Constraint 1

<Filter 2>... -- Constraint 2

)

Specific Filter

Scenario: Show Revenue for "Sofia" regardless of what is selected.

```
Sofia Revenue = CALCULATE(  
    [Total Sales],  
    'Customer'[City] = "Sofia"  
)
```

Remove SPECIFIC Filter (keep others)

```
Sales Ignoring Year = CALCULATE([Total Sales], ALL(Date[Year]))
```

Removing Filters (ALL)

Scenario: Calculate % of Total. We need the Grand Total to ignore slicers.

ALL() function removes filters.

```
All Sales = CALCULATE( [Total Sales], ALL('Sales') )
```

Filter Functions Family

ALL() - Remove ALL filters

```
All Sales = CALCULATE([Total Sales], ALL(Sales))
```

ALLEXCEPT() - Remove all EXCEPT specific

```
Sales Ignore Year = CALCULATE([Total Sales], ALLEXCEPT(Date, Date[City]))
```

→ Keeps City filter, removes everything else

FILTER() - Add custom filter

```
High Value Sales =
```

```
CALCULATE(
```

```
[Total Sales],
```

```
FILTER(Sales, Sales[Amount] > 1000)
```

```
)
```

Calculating % of Total

Common Business Case: What is my share of the pie?

Combine `DIVIDE` and `CALCULATE`.

Revenue % of Total =

`DIVIDE(`

`[Total Revenue],`

`CALCULATE([Total Revenue], ALL('vw_dim_products'[Category]))`

`)`

Live Demo



What is Time Intelligence?

- Analyzing data over time periods (Year, Quarter, Month).
- The most requested feature by CFOs.
- Standard DAX aggregations (SUM) can't do this easily.
- Examples: Year-Over-Year (YoY), Year-to-Date (YTD).

The Golden Rule: Date Table

- To use Time Intelligence, you MUST have a dedicated Date Table (Calendar).
- Do not rely on the automatic date dates in the Sales table.
- Mark as Date Table in Power BI.



Important: You must ensure your Calendar table is marked as a Date table in Power BI

(Right-click table -> Mark as Date table).

Otherwise, Time Intelligence might fail.

SAMEPERIODLASTYEAR

Comparison with the past.

Formula:

Revenue PY = CALCULATE(
[Total Sales],
SAMEPERIODLASTYEAR('Date'[Date]))



Year-Over-Year Growth (YoY)

The formula managers love.

YoY Growth % = DIVIDE (
[Total Sales] - [Revenue PY],
[Revenue PY]



Running Totals (YTD)

TOTALYTD: Accumulates value from Jan 1st to current date.

Revenue YTD =

TOTALYTD(

[Total Revenue],

'vw_fct_sales'[order_date]

)

*Time intelligence works best with a dedicated Date table. This is a simplified example.



Visualizing Time Intelligence

Using a Matrix visual:

- Rows: Year / Month
- Values: Revenue, Revenue PY, Growth %
- Add Conditional Formatting (Red/Green) to Growth.



Live Demo



The Analytics Pane

- The "Magnifying Glass" icon on visuals.
- Adds statistical layers to your charts.
- No coding required!



- **Constant Line:** Show a Target (e.g., Target = 1M).
- **Average Line:** Dynamically calculates the average of visible data.
- Helps users see "Above/Below Average" instantly.



- Predict future values based on historical trends.
- Uses built-in Machine Learning (Exponential Smoothing).
- Requirement: Needs clean time-series data (no gaps).

Time-series data (dates on X-axis)

Numeric values

No gaps in dates

At least 2 seasons of data (recommended)

- Automatically finds "weird" data points (spikes or drops).
- Explains WHY it happened (Smart Narratives).

Custom Visuals (AppSource)

- Power BI has ~30 built-in visuals.
- AppSource has 300+ custom visuals created by the community.
- Warning: Some are paid, some are free. Always check "Power BI Certified" (Blue tick).

Important Warnings:

Custom visuals can slow down reports

Not all work with Publish to Web

Some are paid (\$)

Support may be limited

ALWAYS check "Certified" badge

Test before using in production

- Word Cloud: Text analysis.
- Scroller: Stock market ticker.
- Sankey Chart: Flow of data (Source -> Destination).
- Heatmap Calendar: Visualizing intensity by day.

- Reports are designed for PC (Landscape).
- CEOs use phones (Portrait).
- Mobile Layout View: Lets you rearrange visuals for mobile screens without breaking the desktop version.

Live Demo



Top 5 DAX Mistakes to Avoid

Mixing Types:

- You cannot calculate Text + Number.
- Fix: Use FORMAT() or & correctly.

Text + Number (Use & for text, + for math)

Forgetting Relationships:

- "Why is my number the same for every city?"
- Cause: Usually, there is no active relationship between the tables.

If numbers are repeated everywhere, check your Model View!

Top 5 DAX Mistakes to Avoid

Confusing SUM vs COUNT:

SUM adds up values (Revenue).

COUNT counts lines (Orders).

Check: If your Revenue is showing "1250" (but should be millions), you probably used Count instead of Sum.

Top 5 DAX Mistakes to Avoid

Using Calculated Columns for Aggregations

Wrong: `Total Sales (Column) = SUM(Sales[Amount])`

→ Calculated per row, meaningless!

Right: `Total Sales (Measure) = SUM(Sales[Amount])`

→ Aggregates all filtered rows

Top 5 DAX Mistakes to Avoid

Forgetting DIVIDE for Safety

Wrong: `Margin % = [Profit] / [Revenue]`

→ Error if Revenue = 0

Right: `Margin % = DIVIDE([Profit], [Revenue], 0)`

→ Returns 0 if division by zero

If measure returns BLANK:

- Check relationships (Model View)
- Check filter context (are slicers too restrictive?)
- Check data types (text vs number mismatch)

- Report Strategy & Design
- DAX Measures vs Columns
- Calculate & Context
- Time Intelligence
- Forecasting



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

